Linux Kernel Internals

# Interrupt Handling:

## Multiple Devices on One IRQ Line

*Hardware wire to application · Kernel source · Datasheet-verified · All five IRQ firing cases*

**What this document covers**

- Part I — Foundations: IRQ lines, shared IRQ problem, handler registration
- Part II — The Mechanism: Chain walk, call stack, hardirq/softirq, NAPI, tasklets
- Part III — Case Study: USB + Ethernet on IRQ 19, YouTube + USB file copy
- Part IV — Why the Chain Walk Never Stops Early: all scenarios, spurious IRQs
- Part V — Hardware ↔ Software Boundary: MMIO, ICR register, wire to status bit
- Part VI — All Five IRQ Firing Cases: datasheet-verified, wired-OR, level-triggered

# Table of Contents

Table of Contents ..... 2
1.1 What Is an Interrupt? ..... 3
1.2 The IRQ Line — A Physical Wire ..... 3
1.3 Why Do Devices Share an IRQ? ..... 3
1.4 How a Driver Registers Its Handler ..... 4
2.1 Full Call Stack — From CPU Pin to Device Handler ..... 5
2.2 The Chain Walk — __handle_irq_event_percpu() (v4.10) ..... 5
2.3 What Each Handler Does — USB vs Ethernet ..... 6
2.4 Two Halves: Hardirq and Softirq ..... 7
2.5 The Softirq Priority Bitmask ..... 7
2.6 Ethernet Burst Path — NAPI ..... 8
2.7 USB Burst Path — Tasklet ..... 9
3.1 Setup — Verify It Yourself ..... 10
3.2 Scenario: YouTube + USB File Copy ..... 10
3.3 What Happens Every Time IRQ 19 Fires ..... 10
3.4 The E, U Ordering — How Turns Are Decided ..... 11
3.5 Masking Duration — Real Numbers ..... 11
3.6 Key Code Locations ..... 12
4.1 The Core Reason ..... 13
4.2 The retval State Machine — All Outcomes ..... 13
4.3 Spurious IRQ Tracking — note_interrupt() ..... 14
4.4 Two Masks — A Critical Distinction ..... 14
5.1 The ICR Register — Datasheet Verified ..... 15
5.2 From Wire to Status Register — The Complete Pipeline ..... 16
5.3 The Hardware ↔ Software Boundary — Exactly One Instruction ..... 17
All Five Cases — Summary ..... 21
Questions and Answers — Complete Reference ..... 21

Part I

# Foundations

## 1.1 What Is an Interrupt?

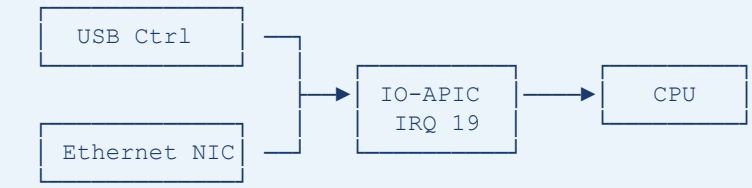
A CPU executing a program does not know when external hardware needs attention — a packet arrived, a USB transfer completed, a key was pressed. Constantly checking every device (polling) wastes CPU cycles. The solution is the interrupt: a hardware signal that tells the CPU to stop, handle the event, and resume exactly where it left off.

> **Analogy**
> You are reading a book. You do not check the door every second. The doorbell rings, you bookmark your page, answer it, and return to the exact line. The doorbell is the interrupt. The bookmark is the saved CPU register state.

## 1.2 The IRQ Line — A Physical Wire

An IRQ (Interrupt Request) line is an electrical signal path managed by the IO-APIC. On PCI, the INT# pin is open-drain (active-LOW). Any device can pull the line LOW to assert an interrupt. The IO-APIC detects this, looks up its Redirection Table, and delivers the vector to the correct CPU.

**Hardware layout — IRQ 19 shared between USB and Ethernet**




```
Both devices share IRQ 19 (open-drain wired-OR).
The CPU sees the wire go LOW — not which device caused it.
```

> **Wired-OR:** On PCI, the INT# line is open-drain. Each device has a transistor that can pull the line LOW. If any device pulls LOW, the line is LOW regardless of what others do. A second device pulling LOW makes no electrical difference — the line is already as low as it can go. This is the wired-OR property.

## 1.3 Why Do Devices Share an IRQ?

The original PC had only 16 IRQ lines. As hardware grew, PCI bus devices share IRQs assigned by firmware. In virtual machines, the hypervisor emulates a legacy IO-APIC and maps multiple virtual devices onto the same IRQ.

| | Bare-Metal Machine | Virtual Machine (Your VM) |
|---|---|---|
| Interrupt type | MSI/MSI-X — unique vector per device | Emulated legacy IO-APIC — shared IRQ |
| Sharing | No sharing, no chain walk | USB + Ethernet share IRQ 19 |
| Code path | pci_enable_msi() succeeds | Falls back to IRQF_SHARED path |
| Performance | Maximum — direct vector delivery | Small overhead from chain walk |

## 1.4 How a Driver Registers Its Handler

When a driver loads it calls request_irq(). IRQF_SHARED tells the kernel: 'I am willing to share this IRQ line.' The kernel allocates an irqaction struct and appends it to the linked list for that IRQ.

**kernel/irq/manage.c — request_threaded_irq()**

```
// kernel/irq/manage.c — request_threaded_irq()

int request_threaded_irq(unsigned int irq,
        irq_handler_t  handler,     // e.g. e1000_intr / uhci_irq
        irq_handler_t  thread_fn,   // threaded handler (optional)
        unsigned long  flags,       // IRQF_SHARED
        const char    *name,
        void          *dev_id)      // unique cookie — identifies THIS device
{
    struct irqaction *action = kzalloc(sizeof(*action), GFP_KERNEL);
    action->handler = handler;
    action->dev_id  = dev_id;
    action->flags   = flags;
    __setup_irq(irq, desc, action); // append to linked list
}
```

After both USB and Ethernet drivers register, the linked list on IRQ 19:

**include/linux/interrupt.h — struct irqaction linked list on IRQ 19**

```
irq_desc[19].action:

  ┌──────────────────────────┐      ┌──────────────────────────┐
  │ irqaction                │      │ irqaction                │
  │  handler : uhci_irq      │─────►│  handler : e1000_intr    │──► NULL
  │  dev_id  : uhci_hcd *    │      │  dev_id  : net_device *  │
  │  name    : 'uhci_hcd'    │      │  name    : 'eth0'        │
  │  flags   : IRQF_SHARED   │      │  flags   : IRQF_SHARED   │
  └──────────────────────────┘      └──────────────────────────┘
   Registered first → called first    Registered second → called second

  Registration order = call order. Cannot change at runtime.
```

Part II

# The Mechanism

## 2.1 Full Call Stack — From CPU Pin to Device Handler

When IRQ 19 fires the CPU saves its full register state and jumps to the interrupt vector. Every function in the path:




## 2.2 The Chain Walk — __handle_irq_event_percpu() (v4.10)

In Linux v4.10, the real chain walk lives in __handle_irq_event_percpu() — note the double underscore. handle_irq_event_percpu() is only a thin wrapper that adds entropy and calls note_interrupt() after the walk.

**kernel/irq/handle.c — __handle_irq_event_percpu() — verified against Intel 8254x SDM**

```
// kernel/irq/handle.c — v4.10  (datasheet-verified)

irqreturn_t __handle_irq_event_percpu(struct irq_desc *desc, unsigned int *flags)
{
    irqreturn_t retval = IRQ_NONE;
    struct irqaction *action;

    record_irq_time(desc);

    for_each_action_of_desc(desc, action) {  // macro — walks until action == NULL
        irqreturn_t res;

        trace_irq_handler_entry(irq, action); // ftrace/perf tracepoint
        res = action->handler(irq, action->dev_id);
        trace_irq_handler_exit(irq, action, res);

        switch (res) {
        case IRQ_WAKE_THREAD:
            __irq_wake_thread(desc, action);
            /* Fall through */
        case IRQ_HANDLED:
            *flags |= action->flags;    // accumulate flags
            retval = IRQ_HANDLED;       // set flag — does NOT stop the loop
            break;                      // breaks switch{} only — NOT the for loop
        // IRQ_NONE: no case — falls to default silently
        }
    }   // loop ends only when action == NULL

    return retval;
}
```

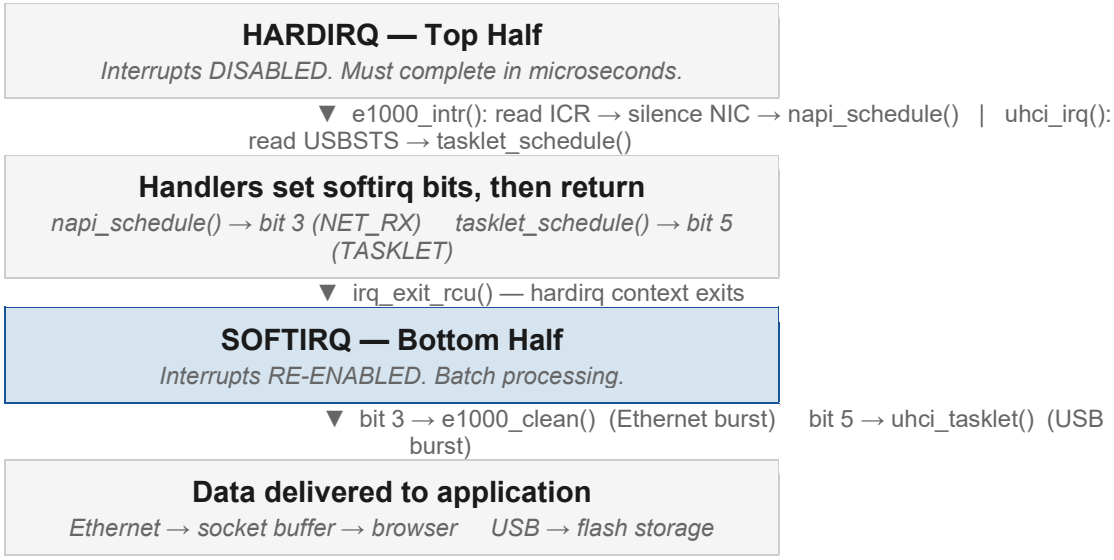
## 2.3 What Each Handler Does — USB vs Ethernet

Each handler reads its own hardware status register. The register either has data (this interrupt is mine) or is empty (not mine). There is no partial execution.

| Step | USB — uhci_irq() | Ethernet — e1000_intr() |
|---|---|---|
| 1. Read register | uhci_readw(uhci, USBSTS) | er32(ICR) — reads ICR at offset 0x000C0h |
| 2. Empty check | if !(status & ~USBSTS_HCH) → IRQ_NONE | if (!icr) → IRQ_NONE |
| 3. Clear register | uhci_writew(uhci, status, USBSTS) | Cleared automatically on read (ICR is read-to-clear) |
| 4. Schedule work | tasklet_schedule() → sets bit 5 | ew32(IMC,~0) → silence NIC napi_schedule() → sets bit 3 |
| 5. Return | IRQ_HANDLED (~250 ns total) | IRQ_HANDLED (~300 ns total) |

## 2.4 Two Halves: Hardirq and Softirq

The interrupt handler does minimum work. Heavy processing is deferred to a softirq — runs after the hardirq exits, with interrupts re-enabled.




## 2.5 The Softirq Priority Bitmask

__do_softirq() scans bits 0→7. Lower bit = higher priority = runs first. This is hardcoded — not configurable.

| Bit | Softirq Name | Who Uses It | Relevant Here |
|---|---|---|---|
| 0 | HI_SOFTIRQ | High-priority tasklets | — |
| 1 | TIMER_SOFTIRQ | Kernel timers | — |
| 2 | NET_TX_SOFTIRQ | Network transmit | — |
| 3 | NET_RX_SOFTIRQ | Ethernet NAPI | ← e1000_clean() — always runs before USB |
| 4 | BLOCK_SOFTIRQ | Block I/O | — |
| 5 | TASKLET_SOFTIRQ | USB tasklets | ← uhci_tasklet() — always runs after Ethernet |
| 6 | SCHED_SOFTIRQ | Scheduler | — |
| 7 | RCU_SOFTIRQ | RCU callbacks | — |

**kernel/softirq.c — __do_softirq()**

```
// kernel/softirq.c — __do_softirq()

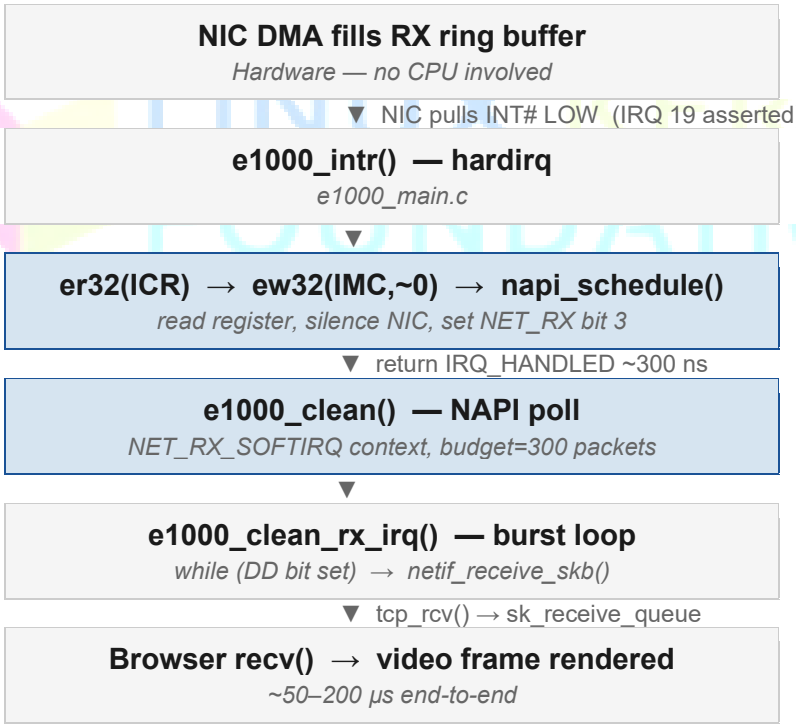
pending = local_softirq_pending();
```

```
// Example: pending = 0b00101000   (bits 3 and 5 set)
//                              ↑↑
//               NET_RX(3)──────┘└──────TASKLET(5)

do {
    if (pending & 1)
        h->action(h);    // bit 3 first → e1000_clean()   [Ethernet]
                         // bit 5 next  → uhci_tasklet()  [USB]
    h++;
    pending >>= 1;       // ← THIS one-bit shift is the Ethernet → USB switch
} while (pending);
```

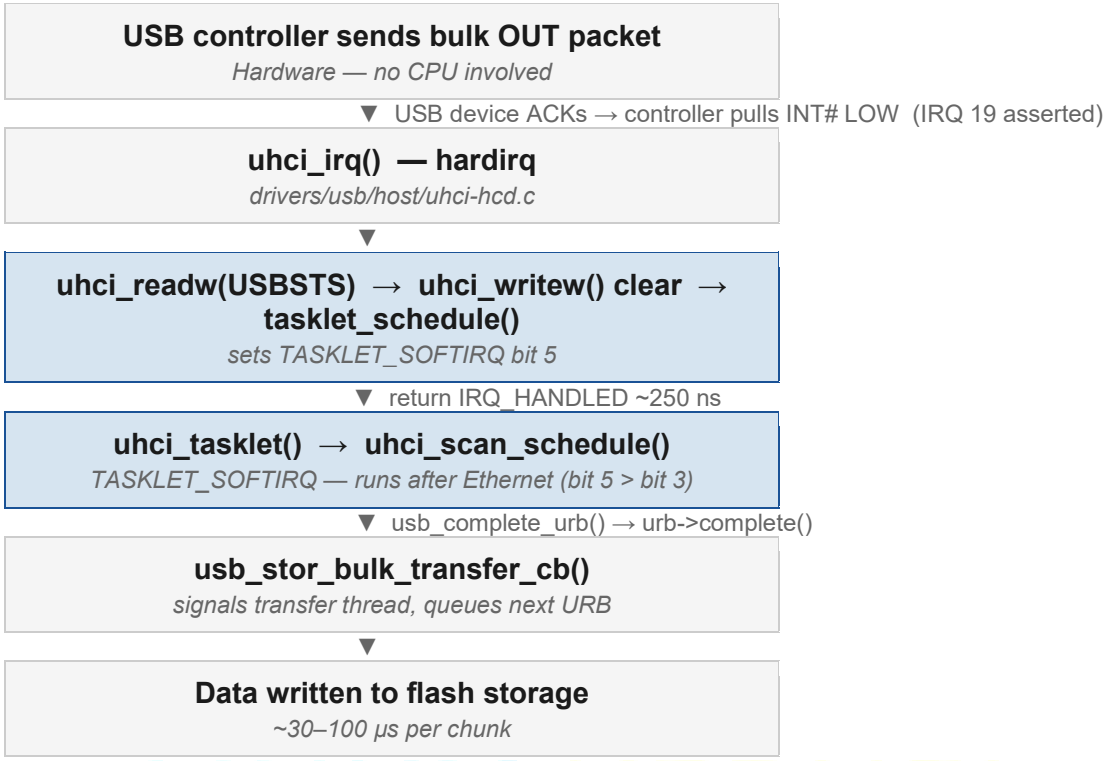
**ksoftirqd:** When softirq work accumulates faster than it can be drained inline, the kernel delegates to ksoftirqd/N — one kernel thread per CPU core (visible in: ps aux | grep ksoftirqd). It is a normal schedulable thread, so the CPU scheduler decides when it runs. This prevents softirq processing from starving user-space processes under heavy network/USB load.

## 2.6 Ethernet Burst Path — NAPI




**Why NAPI drains multiple packets per interrupt:** The NIC does not fire IRQ_HANDLED on every single packet. The ITR (Interrupt Throttle Rate) register sets a minimum gap between interrupts (default ~96 µs). Packets accumulate in the DMA ring during this gap. When the timer expires, one RXT0 interrupt fires and NAPI drains all accumulated packets in one burst (up to budget=300). This is interrupt coalescing — verified by Intel 8254x SDM §13.4.30 (ITR register). Tunable via: /proc/sys/net/core/netdev_budget

## 2.7 USB Burst Path — Tasklet



Part III

# Case Study — Your VM: IRQ 19

## 3.1 Setup — Verify It Yourself

**/proc/interrupts — IRQ 19 in your VM**

```
$ cat /proc/interrupts | grep 19

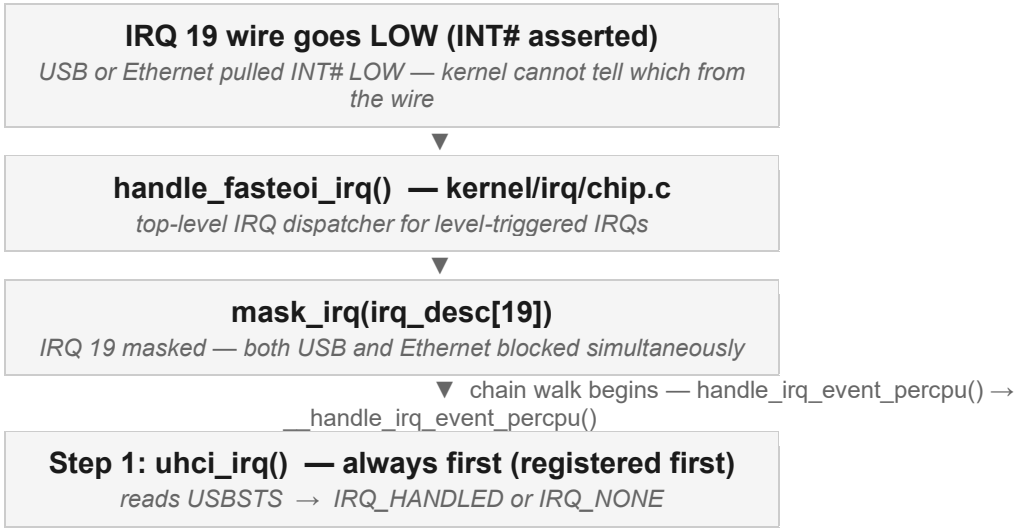
  19:    0    0   IO-APIC  19-fasteoi   uhci_hcd, enp0s3
                                          ^^^^^^^^^^  ^^^^
                                          USB Ctrl    Ethernet  ← same chain
```

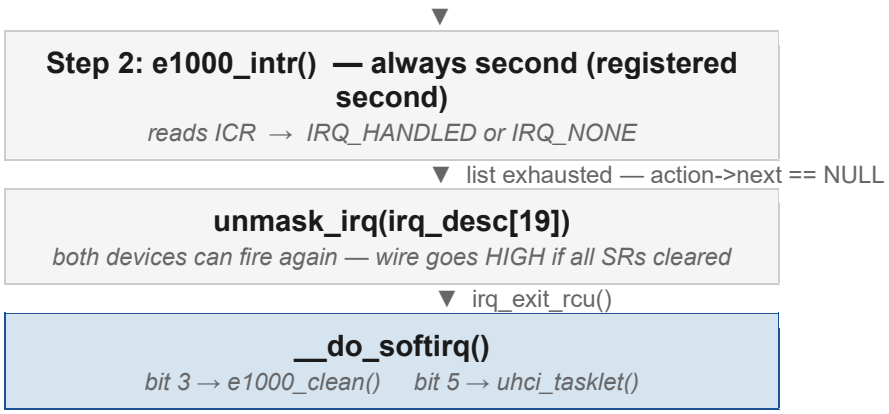
## 3.2 Scenario: YouTube + USB File Copy

You are watching YouTube (continuous network traffic) and copying a large file to USB simultaneously. Both generate interrupts on IRQ 19.

| | YouTube → Ethernet | USB File Copy |
|---|---|---|
| What fires IRQ 19 | NIC DMA fills packet ring buffer | USB controller completes bulk transfer |
| Interrupt rate | ~50,000 IRQs/sec at 1080p | ~8,000 IRQs/sec (USB 2.0 bulk) |
| Transfer unit | Batched packets (NAPI budget=300) | 64 bytes per packet (UHCI/USB 1.1 full-speed bulk max) |
| Data path | NIC → e1000_intr → NAPI → socket → browser | RAM → uhci_irq → tasklet → URB → flash |

## 3.3 What Happens Every Time IRQ 19 Fires






> Chain order is fixed: USB handler is always called before Ethernet, regardless of which device fired. The order is determined by driver registration order, not by who asserted the line.

## 3.4 The E, U Ordering — How Turns Are Decided

| Layer | What It Decides | Rule | Typical Effect |
|---|---|---|---|
| 1 — IRQ chain (ns) | Handler call order | Fixed: USB always before Ethernet (registration order) | USB checks first, Ethernet checks second |
| 2 — Softirq bitmask (µs) | Bottom half order | Fixed: NET_RX bit 3 < TASKLET bit 5 | Ethernet burst always before USB burst |
| 3 — NAPI budget (µs) | When Ethernet yields | After 300 packets, Ethernet stops; USB runs | E×300, U, E×300, U... |
| 4 — IRQ rate (µs–ms) | How often each fires | Hardware data rate: Ethernet ~6× more frequent | E,E,E,U,E,E,E,U pattern |

## 3.5 Masking Duration — Real Numbers

| Event | Duration | vs One 30fps Video Frame (33 ms) |
|---|---|---|
| USB false check (IRQ_NONE) | ~200 ns | 0.0006% — imperceptible |
| Ethernet false check (IRQ_NONE) | ~300 ns | 0.0009% — imperceptible |
| Full chain walk (both handlers) | ~500 ns | 0.0015% — imperceptible |
| Ethernet NAPI burst (300 pkts) | ~50 µs | 0.15% — imperceptible |
| USB tasklet burst (one TD batch) | ~30 µs | 0.09% — imperceptible |

> The waiting is real — ~500 nanoseconds. A single video frame is 33,000,000 nanoseconds. The chain walk overhead is 66,000× shorter than what your eye can perceive.

## 3.6 Key Code Locations

| Function | File | Role |
|---|---|---|
| __handle_irq_event_percpu() | kernel/irq/handle.c | Real chain walk — calls all handlers for IRQ 19 |
| handle_irq_event_percpu() | kernel/irq/handle.c | Thin wrapper — adds entropy + calls note_interrupt() |
| request_threaded_irq() | kernel/irq/manage.c | How drivers join the irqaction chain |
| note_interrupt() | kernel/irq/spurious.c | Tracks spurious IRQs, disables after 99,900 |
| __do_softirq() | kernel/softirq.c | Bitmask scan — runs NET_RX then TASKLET |
| e1000_intr() | drivers/net/ethernet/intel/e1000/e1000_main.c | Ethernet hardirq handler |
| e1000_clean() | drivers/net/ethernet/intel/e1000/e1000_main.c | NAPI poll — burst processing |
| uhci_irq() | drivers/usb/host/uhci-hcd.c | USB hardirq handler |
| uhci_tasklet() | drivers/usb/host/uhci-hcd.c | USB softirq — calls scan_schedule |

Part IV

# Why the Chain Walk Never Stops Early

## 4.1 The Core Reason

IRQ_HANDLED means: 'I fired and I cleared my own hardware register.' It says nothing about whether any other device on the same wire also fired. Stopping the walk early would leave other devices' registers uncleared, keeping the wire LOW, causing an immediate re-interrupt — an IRQ storm.

**include/linux/irq.h — for_each_action_of_desc — no early exit exists**

```
// include/linux/irq.h — v4.10

#define for_each_action_of_desc(desc, act)      \
    for (act = desc->action; act; act = act->next)
//                           ↑
//    stop condition: only act == NULL (list exhausted)
//    NO check for retval == IRQ_HANDLED anywhere in this macro

// In the switch{}: 'break' exits the switch — NOT the for loop.
// The loop ALWAYS walks to the end of the list.
```

**Hypothetical early exit — why it would break:**

- USB handler runs first, finds SR set, returns IRQ_HANDLED
- Hypothetical break: Ethernet handler never called
- Ethernet ICR register never cleared — NIC keeps pulling line LOW
- After unmask: immediate re-interrupt → IRQ storm at 50,000 IRQs/sec
- System destabilises within seconds

## 4.2 The retval State Machine — All Outcomes

**retval state machine — all outcomes from the same loop**

```
irqreturn_t retval = IRQ_NONE;   // starts here every time

// Scenario A: Only Ethernet fired
//   uhci_irq()   → IRQ_NONE    → no case → retval stays IRQ_NONE
//   e1000_intr() → IRQ_HANDLED → retval = IRQ_HANDLED
//   return: IRQ_HANDLED  ✓

// Scenario B: Only USB fired
//   uhci_irq()   → IRQ_HANDLED → retval = IRQ_HANDLED
//   e1000_intr() → IRQ_NONE    → no case → retval stays IRQ_HANDLED
//   return: IRQ_HANDLED  ✓

// Scenario C: BOTH fired simultaneously
//   uhci_irq()   → IRQ_HANDLED → retval = IRQ_HANDLED
```

```
//   e1000_intr() → IRQ_HANDLED → retval = IRQ_HANDLED (written again — idempotent)
//   return: IRQ_HANDLED  ✓  — both softirqs scheduled

// Scenario D: Nobody fired (spurious)
//   uhci_irq()   → IRQ_NONE    → retval stays IRQ_NONE
//   e1000_intr() → IRQ_NONE    → retval stays IRQ_NONE
//   return: IRQ_NONE → note_interrupt() increments spurious counter
```

## 4.3 Spurious IRQ Tracking — note_interrupt()

**kernel/irq/spurious.c — note_interrupt()**

```
// kernel/irq/spurious.c

void note_interrupt(struct irq_desc *desc, irqreturn_t action_ret)
{
    if (action_ret == IRQ_NONE) {
        desc->irqs_unhandled++;

        if (desc->irqs_unhandled > 99900) {
            printk("irq %d: nobody cared!\n", irq);
            // You may see this in dmesg on buggy hardware
            irq_disable(desc);   // IRQ disabled entirely
        }
    } else {
        desc->irqs_unhandled = 0;  // valid IRQ resets the counter
    }
}
```

## 4.4 Two Masks — A Critical Distinction

A common confusion: ew32(IMC, ~0) inside e1000_intr() looks like masking but is completely different from mask_irq(). They operate at different levels and have different scopes.

| | IO-APIC Mask  mask_irq() | NIC Self-Mask  ew32(IMC, ~0) |
|---|---|---|
| What it blocks | IO-APIC delivering IRQ 19 to CPU | e1000 NIC generating any further IRQ |
| Scope | ALL devices on IRQ 19 (USB + Ethernet) | ONLY this e1000 NIC — USB unaffected |
| Duration | ~500 ns — only during chain walk | ~50 µs — entire NAPI poll cycle |
| Set by | Kernel IRQ core (handle_fasteoi_irq) | Ethernet driver (e1000_intr) |
| Removed by | unmask_irq() after chain walk | ew32(IMS, mask) in napi_complete() |
| Purpose | Prevent re-entrant interrupt delivery | Prevent IRQ storm during batch processing |
| Datasheet ref | Intel 8254x SDM §2.6 Interrupts | Intel 8254x SDM §13.4.18 IMC register |

Part V

# Hardware ↔ Software Boundary

## 5.1 The ICR Register — Datasheet Verified

The ICR (Interrupt Cause Read) register is the exact boundary between hardware events and software. Source: Intel PCI/PCI-X Family of Gigabit Ethernet Controllers Software Developer's Manual, Revision 4.0, Section 13.4.17, Table 13-63.

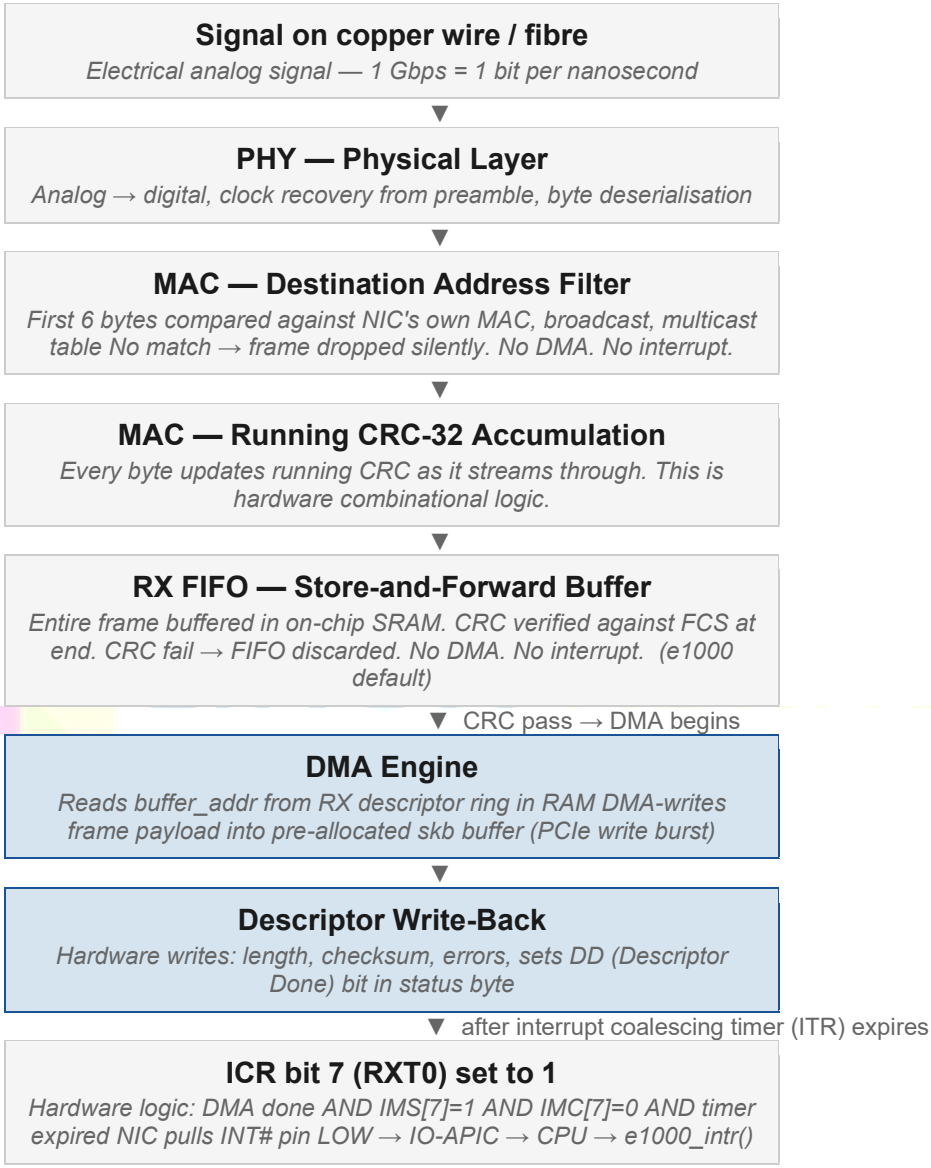
| Bit | Name | Set When | Verified |
|---|---|---|---|
| 0 | TXDW | Hardware processes a TX descriptor with RS bit set | ✓ SDM Table 13-63 |
| 1 | TXQE | Last TX descriptor block used | ✓ SDM Table 13-63 |
| 2 | LSC | Link status changes (cable plug/unplug) | ✓ SDM Table 13-63 |
| 3 | RXSEQ | Receive sequence error | ✓ SDM Table 13-63 |
| 4 | RXDMT0 | RX descriptor minimum threshold reached | ✓ SDM Table 13-63 |
| 6 | RXO | Receiver FIFO overrun | ✓ SDM Table 13-63 |
| 7 | RXT0 | Receiver Timer Interrupt — packet(s) ready | ✓ SDM Table 13-63 |

**Important:** RXT0 (bit 7) is called 'Receiver Timer Interrupt' in the SDM — not 'packet received.' It fires when the interrupt delay timer (RDTR register) expires after a packet arrives, or when a packet count threshold is reached. This is what enables NAPI coalescing — multiple packets accumulate before one interrupt fires.

**Read-to-Clear:** All ICR bits clear on read (SDM Section 13.4.17). Reading er32(ICR) both retrieves the value AND clears all bits atomically. No separate clear step needed — this is why the e1000 driver does not write to ICR after reading it.

## 5.2 From Wire to Status Register — The Complete Pipeline

The ICR bit goes HIGH only after the entire valid frame has been received, CRC-verified, and DMA-written into RAM. Here is every stage:



## 5.3 The Hardware ↔ Software Boundary — Exactly One Instruction

The precise boundary is the readl() call inside er32(ICR). Before it: flip-flops, state machines, electrical signals. After it: a value in a CPU register, manipulated by C code.

**The exact hardware ↔ software boundary — verified against Intel 8254x SDM §13.4.17**

```
// e1000_main.c — e1000_intr()
u32 icr = er32(ICR);

// er32(ICR) expands to:
#define er32(reg)   readl(hw->hw_addr + E1000_ICR)
//                        ↑                ↑
//             kernel virtual addr    offset 0x000C0h (SDM Table 13-2)

// readl() is ONE x86 MOV instruction:
static inline u32 readl(const volatile void __iomem *addr) {
    return *(volatile u32 *)addr;
    // ↑ CPU issues PCIe read transaction
    // ↑ NIC responds with ICR register value
    // ↑ Flip-flop state crosses into CPU register
    // BEFORE this line: hardware
    // AFTER  this line: software
}

// The write direction:
ew32(IMC, ~0);   // writel(0xFFFFFFFF, hw_addr + 0x000D8h)
// ONE x86 MOV instruction → PCIe write → NIC IMC register set
// All 32 interrupt causes masked in the NIC
// BEFORE this line: software intent
// AFTER  this line: hardware state changed
```

**Store-and-Forward (e1000 default — SDM §2.3.3):**

- Entire frame buffered in on-chip SRAM before DMA begins
- CRC verified FIRST — corrupt frames never reach RAM
- ICR bit set only after DMA write is complete and descriptor DD bit written
- This guarantees: when the driver reads er32(ICR), the data is already in RAM

Part VI

# All Five IRQ Firing Cases

When two devices share an IRQ line, five distinct scenarios can occur depending on timing. Each case has a different outcome. These are derived from the hardware behaviour of level-triggered, open-drain, wired-OR PCI interrupts.

## **Case 1** — Both Devices Fire Simultaneously

D1 and D2 both pull the INT# line LOW before the mask is applied. The IO-APIC sees one IRQ assertion. The CPU receives one interrupt. Both status registers are set.

```
t=0   D1 fires → pulls INT# LOW → D1_SR set
t=1   D2 fires → also pulls INT# LOW → D2_SR set
      Wire: already LOW. D2 cannot make it 'more LOW' — wired-OR.
      CPU sees: ONE interrupt delivery.

t=2   mask_irq() — both blocked

t=3   Chain walk:
      H1 → reads D1_SR → SET → IRQ_HANDLED → clears D1_SR
      H2 → reads D2_SR → SET → IRQ_HANDLED → clears D2_SR
      retval written twice — idempotent — both softirqs scheduled

t=4   unmask_irq()
      Wire is now HIGH (both SRs cleared, neither pulling LOW)
      No second interrupt.
```

> Outcome: Both devices served in ONE interrupt delivery. Both return IRQ_HANDLED. This is the happy case — the wired-OR broadcast design works perfectly.

## **Case 2** — D2 Fires While D1's Handler Is Executing

D1 fires first. The mask is applied. During the chain walk, D2 fires and sets its SR. The mask prevents a new interrupt delivery but does not stop D2 from writing its SR. D2's SR is read by H2 in the same chain walk.

```
t=0   D1 fires → INT# LOW → CPU delivers IRQ → mask_irq()

t=1   Chain walk begins:
      H1 → reads D1_SR → SET → IRQ_HANDLED → clears D1_SR

t=2   D2 fires DURING chain walk
      D2 writes D2_SR regardless of mask
      IO-APIC: 'IRQ 19 is masked — hold delivery'
```

```
      D2 pulls INT# LOW — but IO-APIC holds it

t=3   H2 → reads D2_SR → SET → IRQ_HANDLED → clears D2_SR
      (D2 was served because its SR was set before H2 read it)

t=4   unmask_irq()
      H2 cleared D2_SR in step t=3 → D2 releases INT# → wire goes HIGH
      No second interrupt needed — both devices already served.
```

Outcome: Both devices served in ONE interrupt. H1 → IRQ_HANDLED, H2 → IRQ_HANDLED. The mask does not prevent D2 from setting its SR — it only prevents a new CPU interrupt delivery.

**Case 3** — D2 Fires AFTER H2 Already Read Its SR

D1 fires. H1 runs, H2 runs (finds nothing). Then D2 fires — after H2 has already read and found its SR empty. After unmask, the IO-APIC sees the wire LOW again and delivers a fresh interrupt.

```
t=0   D1 fires → INT# LOW → CPU delivers IRQ → mask_irq()

t=1   Chain walk:
      H1 → D1_SR = SET → IRQ_HANDLED → clears
      H2 → D2_SR = 0   → IRQ_NONE  ← H2 looks here, SR empty

t=2   D2 fires AFTER H2 read its SR
      D2_SR set, D2 pulls INT# LOW
      IO-APIC: IRQ 19 masked — holds delivery

t=3   unmask_irq()
      IO-APIC sees wire still LOW (D2 asserting) → delivers FRESH IRQ

t=4   Second chain walk:
      H1 → D1_SR = 0 → IRQ_NONE  (D1 already cleared)
      H2 → D2_SR = SET → IRQ_HANDLED → clears D2_SR ✓
```

Outcome: D2 served correctly, one IRQ cycle late. H2 returning IRQ_NONE in the first pass is correct — it accurately said 'not my device at the time I checked.' This is designed behaviour, not a bug.

## **Case 4** — Clean Case — Only One Device Fires

The most common everyday scenario. One device fires, one does not. The non-firing handler returns IRQ_NONE immediately after reading an empty SR.

```
t=0   D1 fires → INT# LOW → CPU delivers IRQ → mask_irq()

t=1   Chain walk:
      H1 → D1_SR = SET → IRQ_HANDLED → clears D1_SR
      H2 → D2_SR = 0   → IRQ_NONE  (takes ~200 ns to read and return)

t=2   unmask_irq()
      Wire is HIGH (D1 cleared, D2 never asserted)
      No second interrupt.

retval = IRQ_HANDLED (set by H1)
note_interrupt() → irqs_unhandled = 0 (valid IRQ)
```

> Outcome: Clean, quiet, one device served. This accounts for the vast majority of IRQs. The IRQ_NONE from H2 is correct and expected — H2 simply reports that this IRQ was not caused by its device.

## **Case 5** — Spurious IRQ — No Device Claims It

The wire went LOW (triggering an IRQ delivery) but by the time both handlers run, both SRs are 0. The most common hardware cause is a glitch on the INT# line, or a PIC/APIC race where the device de-asserts before the INTA cycle completes.

```
Common causes:

  1. PIC/APIC race (8259A legacy PIC):
     Device asserts IRQ → PIC raises INT to CPU
     Before INTA (acknowledge): device de-asserts
     PIC already committed → sends spurious vector 7 (or 15 for slave)

  2. Hardware glitch:
     Electrical spike on INT# line, resolves before handler runs

  3. Buggy hardware:
     Device auto-clears SR before software can read it

Chain walk result:
  H1 → D1_SR = 0 → IRQ_NONE
  H2 → D2_SR = 0 → IRQ_NONE
  retval = IRQ_NONE

note_interrupt():
  desc->irqs_unhandled++
  if > 99900:
      printk('irq N: nobody cared!')
```

```
irq_disable(desc)    // IRQ line disabled entirely
```

> Outcome: Kernel logs spurious count. After 99,900 consecutive spurious IRQs: line disabled. Real production problem with buggy hardware. You may see 'irq N: nobody cared (try booting with irqpoll)' in dmesg.

## All Five Cases — Summary

| Case | Trigger | H1 Returns | H2 Returns | Outcome |
|---|---|---|---|---|
| 1 — Simultaneous | Both fire before mask | IRQ_HANDLED | IRQ_HANDLED | Both served in ONE IRQ. retval written twice — idempotent. |
| 2 — D2 during walk | D2 fires while H1 running; H2 reads SET SR | IRQ_HANDLED | IRQ_HANDLED | Both served in ONE IRQ. Mask does not prevent SR write. |
| 3 — D2 after H2 | D2 fires after H2 reads empty SR | IRQ_HANDLED | IRQ_NONE (first pass) | D2 served in SECOND IRQ. H2 IRQ_NONE was correct. |
| 4 — One fires | Only D1 fires | IRQ_HANDLED | IRQ_NONE | Only D1 served. Normal everyday case. |
| 5 — Spurious | Glitch / APIC race — no SR set | IRQ_NONE | IRQ_NONE | Nobody claims. note_interrupt() counts. IRQ disabled at 99,900. |

## Questions and Answers — Complete Reference

| Question | Answer |
|---|---|
| Who fires IRQ 19 first? | Whoever pulls the open-drain INT# wire LOW first — pure electrical race, no kernel scheduling. |
| Why open-drain / wired-OR? | PCI INT# is active-LOW open-drain. Any device can pull LOW. Multiple devices pulling LOW simultaneously has no additional effect — the line is already as LOW as possible. |
| Does IRQ_HANDLED stop the chain walk? | No. It sets a flag. The walk always continues to action == NULL (list end). Confirmed by for_each_action_of_desc macro. |
| Can both handlers return IRQ_HANDLED? | Yes — whenever both fire before mask is applied. retval is written twice with same value (idempotent). Both softirqs scheduled. |
| What does IRQ_NONE mean? | 'This interrupt was not caused by my device — my status register was empty when I checked.' Not an error. |
| What does mask_irq() actually block? | IO-APIC delivery of IRQ 19 to the CPU. Blocks ALL devices on that line for ~500 ns. Does NOT stop devices from writing their status registers. |

| Question | Answer |
|---|---|
| What does ew32(IMC, ~0) block? | Only the e1000 NIC from generating further IRQs. USB unaffected. Lasts ~50 µs (NAPI poll). Confirmed by Intel 8254x SDM §13.4.18. |
| Why does ICR clear on read? | Read-to-Clear is a hardware design decision documented in Intel 8254x SDM §13.4.17. Reading er32(ICR) both retrieves and clears all bits atomically. |
| What is RXT0 exactly? | 'Receiver Timer Interrupt' (SDM bit 7). Fires when interrupt delay timer expires after packet(s) arrive. Not per-packet — this enables NAPI coalescing. |
| What triggers a spurious IRQ? | Most commonly: PIC/APIC race where device de-asserts before INTA cycle. Also: electrical glitches, buggy hardware auto-clearing SR. |
| When does the kernel disable an IRQ? | After 99,900 consecutive spurious IRQs (all IRQ_NONE). note_interrupt() in kernel/irq/spurious.c calls irq_disable(desc). |
| Why does the VM share IRQs? | Hypervisor emulates legacy IO-APIC. Physical MSI-X intercepted and remapped to shared virtual IRQ line. |
| Is the data transfer truly continuous? | No. Everything is bursts. Buffers at every layer (DMA ring, socket buffer, page cache, decoder queue) create the illusion of continuity. |

LINUX KERNEL FOUNDATION

## Summary

The Linux shared IRQ mechanism rests on three primitives: a linked list of irqaction structs (the chain), a bitmask of pending softirqs (the priority system), and a packet budget counter (the yield mechanism). These three structures coordinate thousands of IRQs per second between USB and Ethernet — invisible to you, because every layer from DMA ring to socket buffer absorbs and smooths the burstiness below it.

The five cases in Part VI cover every possible timing scenario for two devices on one IRQ line. Cases 1–3 all result in both devices being served correctly — either in one pass or two. Case 4 is the quiet everyday case. Case 5 (spurious) is the only failure mode, and the kernel handles it with a threshold-based disable. The design is elegant in its simplicity and provably correct for all cases.

*Sources: Intel PCI/PCI-X GbE Controllers Software Developer's Manual Rev 4.0 (317453006EN), UHCI Design Guide Rev 1.1 (Intel, March 1996), Linux kernel v4.10 source.*